



UNIVERSITY of ASIA PACIFIC

Department of Computer Science & Engineering (CSE)

Spring-2026

Course Title: Operating System Lab

Course Code: CSE 402

Report Title: Implementation of FCFS, SSTF and SCAN Disk Scheduling Algorithms

Date of Submission: 02-10-2026

Submitted By-

Name : Nacher Uddin Sakib
Registration : 23101093
Semester : 4th Year 1st Semester
Section : B2

Submitted To-

Atia Rahman Orthi
Lecturer, Department of CSE,
University of Asia Pacific

1. Introduction

Disk scheduling decides the order in which pending disk I/O requests are served by the read/write head. A good order reduces the total head movement (seek movement), which lowers seek time and improves overall system performance. This report explains three disk scheduling algorithms — FCFS, SSTF and SCAN (Left and Right) — shows how the head movement is counted step by step, presents the Python code with output, and compares the algorithms.

Input data used:

Algorithm	Request Queue	Initial Head
FCFS	176, 51, 50, 40, 39, 100	35
SSTF	0, 13, 41, 50, 63, 89, 100	46
SCAN (Left & Right)	0, 13, 41, 50, 63, 89, 100	46

2. Algorithm Description

2.1 FCFS (First Come First Served)

FCFS is the simplest disk scheduling algorithm. Requests are served strictly in the order in which they arrive in the queue. The head moves from its current position to the first request, then to the second, and so on, without considering how far each request is. Total seek movement is the sum of the absolute differences between consecutive positions.

2.2 SSTF (Shortest Seek Time First)

SSTF always selects the pending request that is closest to the current head position, i.e. the one with the minimum seek distance. After serving it, the head position is updated and the nearest request is chosen again. This continues until all requests are served.

2.3 SCAN Disk Scheduling

In SCAN, the head moves in one direction, serving all requests on its way, until it reaches the end of the disk. It then reverses direction and serves the remaining requests on the way back, like an elevator. Two variants are used here:

- **SCAN Left:** The head first moves towards cylinder 0 (left), serving all requests smaller than the head, then reverses and serves the requests on the right.
- **SCAN Right:** The head first moves towards the higher end (right), serving all requests larger than the head, then reverses and serves the requests on the left.

3. Step-by-Step Head Movement Calculation

Seek movement for each step = $|To - From|$. The “Total” column shows the running total of head movement.

3.1 FCFS (Head = 35)

Service order: 35 → 176 → 51 → 50 → 40 → 39 → 100

Step	From	To	Calculation	Seek	Total
1	35	176	$ 176 - 35 $	141	141
2	176	51	$ 51 - 176 $	125	266
3	51	50	$ 50 - 51 $	1	267
4	50	40	$ 40 - 50 $	10	277
5	40	39	$ 39 - 40 $	1	278
6	39	100	$ 100 - 39 $	61	339
			Total Seek Movement		339

3.2 SSTF (Head = 46)

Service order: 46 → 50 → 41 → 63 → 89 → 100 → 13 → 0

Step	From	To	Calculation	Seek	Total
1	46	50	$ 50 - 46 $ (nearest: 50 → 4, 41 → 5)	4	4
2	50	41	$ 41 - 50 $ (41 → 9, 63 → 13)	9	13
3	41	63	$ 63 - 41 $ (63 → 22, 13 → 28)	22	35
4	63	89	$ 89 - 63 $ (89 → 26, 13 → 50)	26	61
5	89	100	$ 100 - 89 $ (100 → 11, 13 → 76)	11	72
6	100	13	$ 13 - 100 $ (13 → 87, 0 → 100)	87	159
7	13	0	$ 0 - 13 $	13	172
			Total Seek Movement		172

3.3 SCAN – Left (Head = 46)

Left requests (smaller than 46, in descending order): 41, 13, 0. Right requests: 50, 63, 89, 100.

Service order: 46 → 41 → 13 → 0 → 50 → 63 → 89 → 100

Step	From	To	Calculation	Seek	Total
1	46	41	$ 41 - 46 $	5	5
2	41	13	$ 13 - 41 $	28	33
3	13	0	$ 0 - 13 $	13	46
4	0	50	$ 50 - 0 $ (direction reversed)	50	96
5	50	63	$ 63 - 50 $	13	109
6	63	89	$ 89 - 63 $	26	135
7	89	100	$ 100 - 89 $	11	146
			Total Seek Movement		146

3.4 SCAN – Right (Head = 46)

Right requests (larger than 46, ascending): 50, 63, 89, 100, then end marker 99. Left requests (descending): 41, 13, 0.

Service order: 46 → 50 → 63 → 89 → 100 → 99 → 41 → 13 → 0

Step	From	To	Calculation	Seek	Total
1	46	50	$ 50 - 46 $	4	4
2	50	63	$ 63 - 50 $	13	17
3	63	89	$ 89 - 63 $	26	43
4	89	100	$ 100 - 89 $	11	54
5	100	99	$ 99 - 100 $ (end marker used in code)	1	55
6	99	41	$ 41 - 99 $ (direction reversed)	58	113
7	41	13	$ 13 - 41 $	28	141
8	13	0	$ 0 - 13 $	13	154
			Total Seek Movement		154

Note: As written in the given code, 99 is used as the end-of-disk marker in SCAN Right. Since the head is already at 100, the path $100 \rightarrow 99 \rightarrow 41$ ($1 + 58 = 59$) is equal to going directly $100 \rightarrow 41$ (59), so the total remains 154.

4. Code and Output

4.1 FCFS – Code and Output

```
▶ req_seq = [176,51,50,40,39,100]
  head = 35
  totalseek=0
  for i in req_seq:
    totalseek+=abs(i-head)
    head=i
  print ("Total seek movement =",totalseek)

... Total seek movement = 339
```

Figure 1: FCFS code and output

4.2 SSTF – Code and Output

```
▶ req_seq = [0,13,41,50,63,89,100]
  head = 46
  pending = req_seq.copy ()
  totalseek = 0
  while pending:
    closest = min(pending,key = lambda i: abs (i-head))
    totalseek+=abs(closest-head)
    head=closest
    pending.remove (closest)
  print ("Total seek movement =",totalseek)

... Total seek movement = 172
```

Figure 2: SSTF code and output

4.3 SCAN (Left) – Code and Output

```
For LEFT

req_seq = [0,13,41,50,63,89,100]
head = 46
pending = req_seq.copy ()
pending.sort()
left=[i for i in pending if i<head]
right=[i for i in pending if i>head]
seek_order=left[::-1]+right
totalseek = 0
for i in seek_order:
    totalseek+=abs(i-head)
    head=i
print ("Total seek left =",totalseek)

... Total seek left = 146
```

Figure 3: SCAN Left code and output

4.4 SCAN (Right) – Code and Output

```
For RIGHT

req_seq = [0,13,41,50,63,89,100]
head = 46
pending = req_seq.copy ()
pending.sort()
left=[i for i in pending if i<head]
right=[i for i in pending if i>head]
seek_order=right+[99]+left[::-1]
totalseek = 0
for i in seek_order:
    totalseek+=abs(i-head)
    head=i
print ("Total seek right =",totalseek)

... Total seek right = 154
```

Figure 4: SCAN Right code and output

GitHub Repository link

<https://github.com/Nacher-Uddin-Sakib/Operating-System-laboratory>

5. Discussion

5.1 FCFS

For head position 35 and the queue 176, 51, 50, 40, 39, 100, FCFS gave a total seek movement of 339. The head jumped from 35 to 176 and then back to 51, which created two very long movements (141 and 125). Because FCFS ignores the position of requests, the head makes unnecessary back-and-forth movement even though requests 51, 50, 40 and 39 are close to each other. The algorithm is fair, but its performance is poor.

5.2 SSTF

For head position 46, SSTF gave a total seek movement of 172. It always went to the nearest request, so the first five movements were small (4, 9, 22, 26, 11). However, after reaching 100 the head had to travel back to 13 (87 cylinders), which increased the total. SSTF gives much better performance than FCFS, but requests far from the head may have to wait a long time.

5.3 SCAN

SCAN Left gave a total seek movement of 146, while SCAN Right gave 154. The difference occurs because the head (46) is closer to the left-side requests, so moving left first and then sweeping right is slightly cheaper than going right first. SCAN covers requests in an ordered sweep, so there is no random jumping and no request is ignored for long. Its performance is better than FCFS and, in this example, better than SSTF as well.

6. Advantages and Disadvantages

Algorithm	Advantages	Disadvantages
FCFS	Very simple and easy to implement. Fair – every request is served in arrival order. No starvation.	High total seek movement. Poor performance when requests are scattered. Unnecessary back-and-forth head movement.
SSTF	Much lower seek movement than FCFS. Good average response time. Higher throughput.	Starvation – far requests may wait indefinitely if nearer ones keep arriving. Not fair. Overhead of finding the nearest request every time.
SCAN	Lower and more uniform seek movement. No starvation. Good for heavy load. Easy to predict waiting time.	Requests just behind the head must wait for a full sweep. Head may go to the disk end even if no request is there. Slightly more complex than FCFS.

7. Comparison of the Three Algorithms

Among the three algorithms, FCFS gave the highest total seek movement (339 for 6 requests, an average of 56.50 per request) because it serves requests only in arrival order and makes long back-and-forth jumps. SSTF reduced the movement to 172 (average 24.57 per request) by always choosing the nearest request, but it is not fair and can cause starvation of far-away requests. SCAN gave the lowest movement, 146 for SCAN Left (average 20.86) and 154 for SCAN Right (average 22.00), because the head moves in an ordered sweep in one direction

and then reverses, which also avoids starvation. In terms of complexity, FCFS is the simplest, while SSTF and SCAN need extra processing to select the next request. Overall, FCFS is suitable only for light load, SSTF for moderate load, and SCAN for heavy load. Since FCFS was run on a different request queue (head = 35) from SSTF and SCAN (head = 46), the average seek per request is used for a fair comparison.

8. Conclusion

FCFS produced the highest head movement (339, average 56.50 per request), so it is the least efficient. SSTF reduced the movement to 172, but it can cause starvation. SCAN gave the lowest movement — 146 for SCAN Left and 154 for SCAN Right — while also avoiding starvation and keeping the head movement orderly.

Best algorithm: SCAN is the best among the three, and SCAN Left (total seek = 146) performed best for this data because the head started closer to the left-side requests.